

MedAssist v2

Guide Complet du Projet RAG Avance

Module : IA Generative | **Sujet :** RAG Medical Avance
Technologies : LangChain, ChromaDB, Reranking, Agent Web, Groq
GitHub : github.com/Sliv3er/MedAssist

Pipeline v2 : Documents → Chunking → Embeddings → ChromaDB → Reranking → Agent → Documents / Web → Enrichissement ChromaDB

Fonctionnalite	Description
ChromaDB	Base vectorielle persistante (donnees sauvegardees entre sessions)
Reranking Cross-Encoder	Reordonne les resultats pour meilleure pertinence
Memoire conversation	L'assistant se souvient des echanges precedents
Agent Web (DuckDuckGo)	Si l'info n'est pas dans les docs, recherche automatique web
Enrichissement auto	Les resultats web sont ajoutes a ChromaDB

0. Comment Lancer le Projet (autre PC)

Etape 1 : Cloner le repo

```
git clone https://github.com/Sliv3er/MedAssist.git
cd MedAssist
```

Etape 2 : Installer les dependances

```
pip install -r requirements.txt
```

Etape 3 : Creer le fichier .env

```
copy .env.example .env
```

Modifier .env avec votre cle Groq : GROQ_API_KEY=gsk_votre_cle

Etape 4 : Lancer l'application

```
python -m streamlit run app.py
```

Note : Si 'streamlit' n'est pas reconnu, utiliser `python -m streamlit run app.py`

Ou est la base de donnees ChromaDB ?

Le dossier **chroma_db/** est cree automatiquement au premier lancement dans le repertoire du projet. Il contient les fichiers SQLite et les vecteurs. Ce dossier **persiste** entre les sessions — pas besoin de re-indexer.

Pour reinitialiser la base : supprimer le dossier chroma_db/ et relancer.

Comment voir la recherche web en action ?

1. Lancer l'app avec `python -m streamlit run app.py`
2. Poser une question medicale (ex: 'traitements du diabete') → badge vert '■ Documents'
3. Poser une question HORS des documents (ex: 'C'est quoi le machine learning') → l'agent detecte que les chunks ne sont pas pertinents → lance DuckDuckGo → badge orange '■ Recherche Web' → message vert '■ chunks ajoutes a ChromaDB'
4. Reposer la meme question → maintenant la reponse vient directement de ChromaDB (badge vert '■ Documents') car les resultats web ont ete ajoutes !

1. Vue d'Ensemble du Projet

MedAssist est un assistant medical intelligent base sur le RAG avance (Retrieval-Augmented Generation). Il lit des documents medicaux, les stocke dans ChromaDB, et repond aux questions en se basant sur ces documents. Si l'information n'est pas disponible, il cherche automatiquement sur le web et enrichit sa base.

Architecture des fichiers

Fichier	Role
rag_medical.py	Pipeline RAG : chargement, chunking, ChromaDB, reranking, prompt, memoire
agent.py	Agent intelligent : evaluation pertinence, recherche web, enrichissement ChromaDB
app.py	Interface Streamlit : chat, badges, sidebar, historique
style.css	Theme glassmorphism sombre avec badges colores
data/	3 documents medicaux TXT (diabete, hypertension, antibiotiques)
chroma_db/	Base vectorielle persistante (auto-generee)
requirements.txt	Dependances Python
.env	Cle API Groq (secret)

2. Ce Qui a Change (v1 → v2)

Aspect	Avant (v1)	Apres (v2)	Pourquoi
Base vectorielle	FAISS (en memoire)	ChromaDB (persistant)	Pas de re-indexation au redemarrage
Retrieval	Direct top-3	Top-10 → Reranking → Top-3	Resultats plus pertinents
Memoire	Aucune	5 derniers echanges	Comprend les references
Fallback	'Je ne sais pas'	Agent → DuckDuckGo	Ne reste jamais bloqué
Enrichissement	Non	Auto (web → ChromaDB)	Base de connaissances grandit

3. Pipeline RAG Detaille

Etape 1 : Chargement des documents

Fonction : load_documents() dans rag_medical.py

Parcourt le dossier data/, lit chaque fichier PDF (PyPDFLoader) ou TXT (TextLoader). Retourne une liste d'objets Document avec le contenu et les metadonnees (nom du fichier, page).

Etape 2 : Chunking (Decoupage)

Fonction : split_documents() dans rag_medical.py

Parametres : CHUNK_SIZE=800, CHUNK_OVERLAP=150

Le RecursiveCharacterTextSplitter decoupe intelligemment : d'abord sur les paragraphes, puis les lignes, puis les phrases. L'overlap de 150 caracteres evite de couper une information entre deux chunks. Resultat : 3 documents → 16 chunks.

Etape 3 : Embeddings + ChromaDB

Fonction : create_vectorstore() dans rag_medical.py

Modele : all-MiniLM-L6-v2 (384 dimensions)

Chaque chunk est transforme en vecteur numerique (384 nombres) qui capture le sens du texte. Ces vecteurs sont stockes dans ChromaDB (dossier chroma_db/) — une base persistante. Au prochain lancement, la base est chargee depuis le disque sans re-indexation.

Difference cle : FAISS stocke tout en RAM (perdu au redemarrage). ChromaDB stocke sur disque dans chroma_db/ (persist entre sessions).

Etape 4 : Retrieval + Reranking

Fonction : create_retriever() dans rag_medical.py

Le retrieval se fait en 2 phases :

Phase 1 — Retrieval large : ChromaDB cherche les 10 chunks les plus proches par similarite cosinus (rapide mais approximatif).

Phase 2 — Reranking precis : Le Cross-Encoder (ms-marco-MiniLM-L-6-v2) prend la question ET chaque chunk ensemble, les evalue, et ne garde que les 3 meilleurs.

Analogie : Un recruteur recoit 100 CV (base), en pre-selectionne 10 (cosinus), puis les lit en detail pour garder les 3 meilleurs (cross-encoder).

Etape 5 : Agent (Decision)

Fonction : agent_answer() dans agent.py

L'agent demande au LLM : 'Ces chunks sont-ils pertinents pour la question ? OUI/NON'

- Si OUI → repond depuis les documents (badge vert)
- Si NON → recherche web DuckDuckGo → repond → enrichit ChromaDB (badge orange)

Etape 6 : Memoire de conversation

Fonction : format_history() dans rag_medical.py

Les 5 derniers echanges (question + reponse) sont injectes dans le prompt. Le LLM comprend les references aux messages precedents. Ex: 'Et les effets secondaires ?' → comprend qu'on parle du medicament mentionne avant.

Etape 7 : Enrichissement automatique

Fonction : add_to_vectorstore() dans rag_medical.py

Quand l'agent fait une recherche web, chaque resultat (titre + contenu + URL) est decoupe en chunks et ajoute

a ChromaDB. La prochaine fois, la meme question sera repondue directement depuis la base locale.

4. Toutes les Fonctions du Projet

Fonction	Fichier	Role	Nouveau
load_documents()	rag_medical.py	Charge les fichiers PDF/TXT de data/	Non
split_documents()	rag_medical.py	Decoupe en chunks (800 car, overlap 150)	Non
get_embeddings()	rag_medical.py	Retourne le modele HuggingFace	Non
create_vectorstore()	rag_medical.py	Cree/charge la base ChromaDB persistante	OUI
create_retriever()	rag_medical.py	Retriever + CrossEncoder reranking	OUI
add_to_vectorstore()	rag_medical.py	Ajoute du texte web dans ChromaDB	OUI
build_prompt()	rag_medical.py	Template avec historique conversation	Modifie
format_context()	rag_medical.py	Formate les chunks en texte lisible	Non
format_history()	rag_medical.py	Formate les 5 derniers echanges	OUI
answer_question()	rag_medical.py	Retrieval + LLM + reponse	Modifie
init_pipeline()	rag_medical.py	Initialise tout le pipeline	OUI
evaluate_context_relevance()	agent.py	LLM evalue si chunks pertinents	OUI
web_search()	agent.py	Recherche DuckDuckGo	OUI
format_web_results()	agent.py	Formate resultats web en texte	OUI
agent_answer()	agent.py	Logique agent : docs ou web	OUI
init_rag()	app.py	Init pipeline dans Streamlit	Modifie
render_sidebar()	app.py	Sidebar avec stats et pipeline v2	Modifie
render_chat()	app.py	Chat avec badges et enrichissement	Modifie

5. Questions Techniques du Professeur

Q : C'est quoi la difference entre FAISS et ChromaDB ?

FAISS stocke les vecteurs en memoire (RAM) — quand on eteint le programme, tout est perdu et il faut re-indexer. ChromaDB est une vraie base de donnees vectorielle qui persiste sur le disque dans le dossier `chroma_db/`. Au prochain lancement, on charge directement la base existante.

Q : C'est quoi le reranking ?

C'est une deuxieme phase de filtrage apres le retrieval. Le retriever classique (similarite cosinus) est rapide mais approximatif. Le Cross-Encoder prend la question et chaque chunk ensemble, les lit vraiment, et donne un score plus precis. On recupere 10 candidats, le Cross-Encoder les reordonne, et on garde les 3 meilleurs.

Q : C'est quoi un Cross-Encoder ?

C'est un modele qui prend deux textes en entree (question + document) et donne un score de similarite. Contrairement aux embeddings qui encodent les textes separement, le Cross-Encoder les traite ensemble — c'est plus lent mais beaucoup plus precis.

Q : Pourquoi 10 puis 3 et pas directement 3 ?

Si on prend directement 3, on risque de rater des chunks pertinents qui seraient classes 4eme ou 5eme par la similarite cosinus. En prenant 10 puis en reranking, on a plus de chances de trouver les 3 meilleurs vrais resultats.

Q : Comment fonctionne la memoire de conversation ?

On garde les 5 derniers echanges (messages utilisateur + reponses) et on les injecte dans le prompt. Le LLM peut ainsi comprendre les pronoms et les references. Ex: 'Et les effets secondaires ?' → comprend qu'on parle du medicament mentionne avant.

Q : Comment l'agent decide de chercher sur le web ?

L'agent envoie les chunks au LLM avec la question et lui demande 'est-ce pertinent ? OUI ou NON'. Si NON (les chunks ne parlent pas du sujet), l'agent lance DuckDuckGo automatiquement.

Q : Pourquoi DuckDuckGo et pas Google ?

DuckDuckGo est gratuit et ne necessite aucune cle API. Google Search API est payant. Pour un projet academique, DuckDuckGo est parfait.

Q : Comment fonctionne l'enrichissement ?

Quand l'agent fait une recherche web, il prend chaque resultat (titre + contenu + URL), le decoupe en chunks, et l'ajoute dans ChromaDB. La prochaine fois, la meme question sera repondue directement sans recherche web.

Q : C'est quoi le ContextualCompressionRetriever ?

C'est un composant LangChain qui combine un retriever de base (ChromaDB) avec un compresseur (Cross-Encoder). Il recupere d'abord les documents, puis les passe au compresseur qui les filtre et reordonne.

Q : Que se passe-t-il si le web ne trouve rien ?

Si DuckDuckGo ne retourne rien (ou erreur reseau), l'agent revient aux chunks de ChromaDB meme s'ils ne sont pas parfaits, et le LLM fait de son mieux. Il n'y a jamais de blocage.

Q : Est-ce que le LLM apprend de vos documents ?

NON. Le LLM (Llama 3.3) est fige. On lui donne les documents en contexte a chaque question, mais ses poids ne changent pas. C'est la difference entre RAG et fine-tuning.

Q : Difference entre RAG et fine-tuning ?

RAG : on donne les docs au moment de la question, le modele ne change pas. Fine-tuning : on reentraine le modele sur nos donnees. RAG est prefere quand les donnees changent souvent.

Q : Pourquoi temperature=0 ?

Temperature=0 rend le modele deterministe : pas de creativite ni d'invention. Pour le medical, on veut des reponses fiables et reproductibles.

Q : C'est quoi les tokens ?

Unite de texte traitee par le LLM (environ un mot). Les LLM ont une limite (ex: 8192 tokens), d'ou l'importance du chunking et de la limite de 5 echanges dans l'historique.

Q : Comment ameliorer le systeme ?

1) Ajouter un upload de documents dans l'interface. 2) Supporter plus de formats (DOCX, HTML). 3) Utiliser un LLM local. 4) Ajouter une evaluation automatique des reponses. 5) Deployer sur le cloud.

6. Vocabulaire a Maitriser

Terme	Definition
RAG	Retrieval-Augmented Generation — combiner recherche + generation
LLM	Large Language Model — gros modele de langage (Llama, GPT)
Embedding	Vecteur numerique representant le sens d'un texte (384 dim.)
Chunk	Morceau de texte decoupe d'un document (800 car.)
Overlap	Chevauchement entre chunks consecutifs (150 car.)
ChromaDB	Base vectorielle persistante (stockee sur disque)
FAISS	Facebook AI Similarity Search (stocke en RAM uniquement)
Cross-Encoder	Modele qui evalue question+document ensemble pour reranking
Reranking	Reordonnement des resultats pour meilleure pertinence
Retriever	Composant qui cherche les chunks pertinents
Agent	Systeme qui decide quelle action effectuer (docs ou web)
Grading	Evaluation de la pertinence des chunks par le LLM
Enrichissement	Ajout de nouveaux contenus (web) dans la base vectorielle
PromptTemplate	Template structure pour formater les requetes au LLM
Similarite cosinus	Mesure de ressemblance entre deux vecteurs
Hallucination	Quand un LLM invente une reponse fausse
Temperature	Parametre de creativite (0=deterministe, 1=creatif)
Persistence	Capacite a sauvegarder les donnees entre les sessions

Conseil : Parle naturellement. Si le prof demande quelque chose, ne recite pas — explique avec tes mots. Utilise des analogies : 'le reranking c'est comme un recruteur qui pre-selectionne 10 CV puis les lit en detail pour garder les 3 meilleurs'.